

OSSP PRACTICAL PROGRAM Q4: -

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Extend the program to demonstrate how improper synchronization can lead to zombie processes. Use appropriate system calls to prevent zombie processes.

Code: -

```
#include
<stdio.h>
#include
<stdlib.h>
#include
<unistd.h>
#include
<sys/types.h>
#include
<sys/wait.h> int
main() {

    pid_
    t
    pid[
    3];
    int i,
    statu
    s;

    for (i = 0; i < 3; i++) {
        pid[i] = fork();
        if (pid[i] < 0)
            { perror("for
```

```

        k failed");
        exit(1);
    }
    if (pid[i] == 0) {
        printf("Child %d started\n",
            i + 1); printf("PID : %d\n",
            getpid());
        printf("PPID : %d\n",
            getppid()); sleep(i + 2);

        printf("Child %d
            finished\n\n", i + 1); exit(0)
    }
}

printf("Parent PID: %d\n\n",
getpid()); printf("Using
wait():\n");
or (i = 0; i < 2; i++) {
    wait(sstatus);

    printf("A child process terminated.\n");
}

printf("\nUsing waitpid():\n");
waitpid(pid[2], sstatus, 0);

printf("Child with PID %d terminated.\n",
pid[2]); printf("\nAll child processes
completed.\n"); return 0;
}

```

OUTPUT: -

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int i, status;

    // Create 3 child processes
    for (i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] < 0) {
            printf("Fork failed!\n");
            exit(1);
        }
    }
}
```

^G Get Help	^O WriteOut	^R Read File	^Y Prev Pg	^K Cut Text	^C Cur Pos
^X Exit	^J Justify	^W Where is	^V Next Pg	^U UnCut Text	^T To Spell

apple@apples-MacBook-Pro-2 practical % ./practicalq4

Parent waiting using wait()

Child 1 started

PID = 980

PPID = 978

Child 2 started

PID = 981

PPID = 978

Child 3 started

PID = 982

PPID = 978

Child 1 completed

A child process terminated.

Child 2 completed

A child process terminated.

Child 3 completed

A child process terminated.

All child processes completed using wait().